

# 011: StepPPO-v2 改进方案

## 目标

本文记录 StepPPO-v2 的第一轮改进。改进目标不是重新定义 StepPPO，而是在当前 v1 的不稳定性和运行时间诊断基础上，做两个低风险修正：降低 value loss 对 actor backbone 的干扰，并减少 old-log-probability 阶段的重复 forward 开销。

## 问题回顾

StepPPO-v1 的主要问题包括：

- valid action ratio 明显低于 GiGPO，说明 action format 退化。
- response clipping 更高，导致 rollout generation 和后续 forward 更慢。
- value loss 较大且波动，且默认会回传到 shared actor backbone。
- final advantage 由两个 whitened signals 平均得到，可能削弱 policy-gradient scale。
- old-log-probability 阶段额外计算 state value，带来明显运行开销。

## 改进 1: Detach Value Backbone

原 v1 配置为

```
detach_value_backbone=False,    value_loss_coef=0.1.
```

这会让 noisy value regression 直接更新 actor backbone。v2 改为

```
detach_value_backbone=True,     value_loss_coef=0.03.
```

这样 value loss 主要训练 value head，而不再直接牵引语言模型 backbone。policy loss 仍然正常更新 actor。该改动的目标是减少 invalid action、response clipping 和 gradient norm 震荡。

## 改进 2: Final Advantage Whitening

v1 的 advantage 为

$$A_t^{\text{final}, \text{v1}} = \frac{\hat{A}_t^{\text{epi}} + w \hat{A}_t^{\text{step}}}{1 + w}.$$

由于  $w = 1$  时相当于两个 normalized signals 的平均，final advantage 的尺度可能偏小。v2 保留该混合形式，但在混合后增加可配置 whitening：

$$\bar{A}_t = \frac{\hat{A}_t^{\text{epi}} + w \hat{A}_t^{\text{step}}}{1 + w}, \quad A_t^{\text{final}, \text{v2}} = \text{Whiten}(\bar{A}_t).$$

对应配置为

`algorithm.step_ppo.normalize_final_advantage=True`.

默认配置中该开关保持关闭，以保证 v1/v0 的行为不变。

### 改进 3: Old Log Probability 与 Value 单 Forward

原实现中，当 value head 启用时，old-log-probability 阶段先调用普通 actor forward 计算 token log probability，再额外调用一次 value forward 计算 old state values。v2 将该路径改为调用已有的 combined forward：

$$(\log \pi_{\theta_{\text{old}}}, V_t^{\text{old}}) = f_{\theta_{\text{old}}, \psi_{\text{old}}}(s_t, a_t).$$

这不改变算法输出，只减少重复计算。预期主要降低 `timing_s/old_log_prob`。

### 改进 4: Episode Advantage 与 GiGPO 对齐

GiGPO 的 episode advantage 不是对整个 batch 做全局 whitening，而是在同一个 initial task group 内做 relative normalization。设一个训练 batch 中共有  $N$  个 action/step samples，第  $i$  个 sample 的符号定义如下：

- $u_i$ : episode group id，对应代码中的 `uid`。同一个  $u_i$  表示同一个 initial task/prompt 的多次 rollout。
- $\tau_i$ : trajectory id，对应代码中的 `traj\_uid`。同一个  $\tau_i$  表示同一条 sampled episode 中的不同 step samples。
- $\ell \in \{1, \dots, L\}$ : 当前 action response 的 token 位置。
- $m_{i,\ell} \in \{0, 1\}$ : response mask，只在有效 action tokens 上参与 policy loss。
- $r_{i,\ell}^{\text{tok}}$ : 第  $i$  个 sample 在 token  $\ell$  上的 reward 张量值。
- $S_i$ : 第  $i$  个 sample 对应的 episode-level scalar score，定义为

$$S_i = \sum_{\ell=1}^L r_{i,\ell}^{\text{tok}}.$$

对每个 group  $u$ ，定义当前 GiGPO 代码默认使用的 row-level group set：

$$\mathcal{I}_u = \{i \in \{1, \dots, N\} : u_i = u\}, \quad n_u = |\mathcal{I}_u|.$$

当 `compute\_mean\_std\_cross\_steps=True` 时， $\mathcal{I}_u$  包含该 group 下所有 step rows。因此如果一条 trajectory 更长，它会在 episode advantage 的 group statistics 中贡献更多 rows。这是当前 GiGPO 实现的默认行为。

若  $n_u > 1$ ，定义 group mean 和 group standard deviation 为

$$\mu_u = \frac{1}{n_u} \sum_{j \in \mathcal{I}_u} S_j,$$

$$\sigma_u = \sqrt{\frac{1}{n_u - 1} \sum_{j \in \mathcal{I}_u} (S_j - \mu_u)^2}.$$

这里  $\sigma_u$  对齐当前实现中 `torch.std` 的默认 sample standard deviation。若  $n_u = 1$ , 当前 GiGPO episode 实现采用

$$\mu_u = 0, \quad \sigma_u = 1.$$

GiGPO 支持两种 episode normalization mode。对于 `mean\_norm`, 只做 group mean centering:

$$A_i^{\text{epi}} = S_i - \mu_{u_i}.$$

对于 `mean\_std\_norm`, 同时除以 group standard deviation:

$$A_i^{\text{epi}} = \frac{S_i - \mu_{u_i}}{\sigma_{u_i} + \epsilon}.$$

最后将该 scalar episode advantage broadcast 到当前 action 的有效 response tokens:

$$A_{i,\ell}^{\text{epi}} = A_i^{\text{epi}} m_{i,\ell}.$$

为了让 StepPPO 的 episode term 与 GiGPO 对齐, v2 不应再对  $A_i^{\text{epi}}$  做额外的 batch-level whitening:

$$A_i^{\text{epi, StepPPO-v2}} = A_i^{\text{epi, GiGPO}}, \quad \text{no extra } \text{Whiten}_{\mathcal{B}}(\cdot).$$

WebShop 的 GiGPO 脚本默认使用 `mode=mean\_norm`, 因此对应的对齐版本应默认采用

$$A_i^{\text{epi, align}} = S_i - \mu_{u_i}.$$

需要特别区分的是, 一个更严格的 trajectory-level episode definition 可以先对同一  $(u, \tau)$  去重:

$$\tilde{\mathcal{I}}_u = \{\min\{i : u_i = u, \tau_i = \tau\} : \tau \in \mathcal{T}_u\},$$

再用  $\tilde{\mathcal{I}}_u$  计算  $\mu_u$  和  $\sigma_u$ 。这个版本在语义上更接近“每条 episode 只贡献一次”, 但它不等价于当前 GiGPO 默认代码行为。因此若实验目标是严格复现 GiGPO baseline, 应优先采用 row-level group set  $\mathcal{I}_u$ ; 若目标是更干净的 StepPPO 变体, 可以把 trajectory-level 去重作为单独 ablation。

## 新增脚本

正式 StepPPO-v2 脚本为 `exps/run_webshop_step_ppo_v2_4gpu_paper_align_simple.sh`。旧的 `v2\_stable` 脚本已删除, 后续只保留当前确定版 v2 入口。核心覆盖配置为:

```
algorithm.step_ppo.step_advantage_w=0.5
algorithm.step_ppo.episode_mode=mean_norm
algorithm.step_ppo.final_advantage_mode=direct
algorithm.step_ppo.normalize_episode_advantage=False
algorithm.step_ppo.normalize_step_advantage=True
algorithm.step_ppo.normalize_final_advantage=False
actor_rollout_ref.actor.value_head.detach_value_backbone=False
actor_rollout_ref.actor.value_head.value_loss_coef=0.03
actor_rollout_ref.actor.ppo_micro_batch_size_per_gpu=8
trainer.experiment_name=step_ppo_v2_qwen2.5_1.5b_4gpu_paper_align_full_e250
```

## 建议运行命令

单 seed 首先运行:

```
bash exps/run_webshop_step_ppo_v2_4gpu_paper_align_simple.sh env.seed=2026
```

如需 debug, 可以额外覆盖 `trainer.total_epochs` 或 `trainer.test_freq`, 但正式 wall-clock 对比不应改变 validation protocol。

## 需要观察的指标

v2 是否有效，优先看以下指标：

- `episode/valid_action_ratio` 是否接近 GiGPO。
- `response_length/clip_ratio` 是否下降。
- `actor/grad_norm` 是否低于 v1 并在后期下降。
- `actor/value_loss` 是否下降且不压制 policy learning。
- `timing_s/old_log_prob` 是否低于早期 v1 实现。
- `timing_s/update_actor` 是否因 micro-batch 对齐 GiGPO 而下降。
- validation task score、success rate、text score 是否优于 v1。

## 结论

StepPPO-v2 是 GiGPO-aligned 的 StepPPO 版本。它保留 StepPPO 的核心定义：step-wise GAE、shared actor value head、episode-level relative correction；同时将 episode advantage 与 GiGPO 的 `mean\_norm` 对齐，将 final advantage 改为 direct addition，降低 value loss coefficient，并将 actor PPO micro-batch 对齐 GiGPO。value loss 仍更新 shared actor backbone，即 `detach\_value\_backbone=False`。